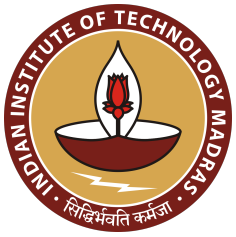

Indian Institute of Technology, Madras

Department of Computer Science

CS4900 - UGRC Report

Arya Krishnamurthy

Supervisor: Dr. Srinivas N.G.

A report submitted as a requirement of
the Undergraduate Research Course (CS4900) offered by
the department of Computer Science and Engineering
at *IIT Madras*

May 5, 2026

Abstract

Packet-processing applications in middleboxes such as firewalls and load balancers are commonly written as sequential C programs that are typically loopless and parse incoming packets by extracting header fields and branching on their values. However, it is hard to build automatic optimisers directly from this representation.

This project presents a synthesis-based compiler that automatically translates sequential C-like packet parsing code into an equivalent finite-state machine (FSM). The compiler involves a Counterexample-Guided Inductive Synthesis (CEGIS) loop using the Z3 SMT solver, and produces a tabular FSM that maps to P4, a language for programmable network hardware. The CEGIS verifier checks all possible input packets to confirm that the synthesised FSM produces the same ACCEPT/REJECT verdict and extracts the same set of header fields as the original code.

The FSM representation may be used for the following optimisations.

First, **payload parking**: The generated P4 code can be used to program hardware to extract and forward only the relevant headers, avoiding unnecessary movement of packet payloads through middleboxes that act only on header fields.

Second, **batch processing**: The synthesised FSM captures the control flow structure that arises due to parsing. When no other branches exist in the processing code, packets in the same parse state follow identical execution until the next branch point in the parse graph and can therefore be grouped and processed together using vector instructions. Even when additional branches exist (for example, due to TTL checks or lookup-dependent logic), further analyses may be done to handle divergence due to factors other than parsing.

The FSM thus provides a foundation for further analyses targeting hardware-accelerated packet processing.

Contents

1. Introduction

1.1 Background

A network packet is a stream of bytes which consists of a sequence of headers and a payload. The sequence of headers is not the same for all packets as multiple protocols may be followed in a network. Parsing involves identifying the sequence of headers in a packet and is the first step of any packet processing pipeline.

Parsing involves reading (extracting) headers one after another each of which indicates which header to continue parsing next or finish parsing and accept or reject the packet. It can be seen that parsing is simulating a FSM where the field extracted in each state indicates which state to proceed to next.

Packet processing code today is written using C(for software packet processors) or using P4(for hardware packet processors). Parsers in P4 directly map to FSMs while the control flow in C code implicitly encodes a FSM. A packet-processing program in C and its P4 equivalent are shown in Figure 1.1. Only the parsing part is shown for brevity. It can be observed that P4 has a separate ingress processing block, while in C the processing is inline with the parser. However, we may do prior analyses to split the parsing and processing parts, and so in this project we consider only the parsing part of the C code.

1.2 Motivation

Sequential C code cannot directly be used to program hardware and it is also not the best representation for analyses involving control flow divergence due to parsing. A FSM or the mapped P4 version achieve both these objectives better. It would thus be beneficial to automatically convert sequential C-like code to an equivalent FSM represented in P4. We want both the synthesised FSM and the input C-like code to extract the same set of headers for any given packet and make the same final decision of accepting or rejecting the packet.

1.3 Objectives

1. Study program synthesis techniques commonly used in formal verification
2. Design a restricted C-like DSL which represents realistic parsers
3. Build a pipeline to automatically convert the input code to an equivalent FSM in P4 using program synthesis

1.4 Scope of the Study

The DSL is restricted in some ways: it doesn't allow loops or arbitrary branch conditions, and is purely sequential with no support for function calls. It also doesn't support lookahead: only the fields of headers which are extracted can be branched on and variable length fields are not allowed. However, these features are very rare in parsers and our DSL would be useful for many real-life network parsers.

```
struct ethernet_t {
    unsigned char  dst[6];
    unsigned char  src[6];
    unsigned short ether_type;
};

struct ipv4_t { /* fields */ };
struct arp_t  { /* fields */ };

int process(unsigned char *ptr) {
    struct ethernet_t ethhdr;
    struct ipv4_t     ipv4;
    struct arp_t      arp;

    ethhdr = *((struct ethernet_t
    *)ptr);
    ptr   += sizeof(struct
    ethernet_t);

    if (ethhdr.ether_type ==
    0x0800) {
        ipv4 = *((struct ipv4_t
    *)ptr);
        ptr += sizeof(struct
    ipv4_t);
        /* further processing */
        return ACCEPT;
    } else if (ethhdr.ether_type
                == 0x0806) {
        arp = *((struct arp_t
    *)ptr);
        ptr += sizeof(struct
    arp_t);
        return REJECT;
    }
    /*further processing*/
    return ACCEPT;
}
```

(a) A packet processing program in C.

```
header ethernet_t {
    bit<48> dst_addr;
    bit<48> src_addr;
    bit<16> ether_type;
}

header ipv4_t { /* fields */ }
header arp_t  { /* fields */ }

struct headers_t {
    ethernet_t ethernet;
    ipv4_t     ipv4;
    arp_t      arp;
}

parser my_parser(packet_in packet,
                 out headers_t hd,
                 ...) {
    state start {

    packet.extract(hd.ethernet);
        transition select(

    hd.ethernet.ether_type) {
            0x0800:  parse_ipv4;
            0x0806:  parse_arp;
            default: accept;
        }
    }
    state parse_ipv4 {
        packet.extract(hd.ipv4);
        transition accept;
    }
    state parse_arp {
        packet.extract(hd.arp);
        transition reject;
    }
}
/*further processing*/
```

(b) Equivalent program in P4.

Figure 1.1: The same packet-processing program shown in both C and P4. Unnecessary details omitted for clarity.

2. Literature Review

2.1 Z3

Z3 [6] is a Satisfiability Modulo Theories (SMT) solver developed by Microsoft. SMT extends boolean satisfiability (SAT) by allowing constraints over richer theories such as bit-vectors, integers and arrays. Internally, Z3 uses CDCL (Conflict-Driven Clause Learning) to decide satisfiability of formulas in these theories. DPLL (Davis-Putnam-Logemann-Loveland) is a classical backtracking algorithm for SAT that combines case splitting with unit propagation. CDCL extends DPLL by analysing each conflict during the search and learning a new clause from it, which prunes the future search by preventing revisits to similar dead-ends. We used Z3's Python API (Z3py) to encode our specifications and the Z3 documentation was referred to often throughout the project.

2.2 CEGIS

The Syntax-Guided Synthesis paper [1] and the Search-Based Program Synthesis primer [2] provide a simple walkthrough of CEGIS. They served as our introduction to CEGIS and provided the foundations for understanding the ParserHawk paper. To get more familiar with these concepts, we built a small CEGIS-based toy DSL which infers a function from a logical specification. The code for this is present in our GitHub repository [5].

2.3 ParserHawk

ParserHawk [4] is the main reference for our project. It solves a similar synthesis problem for parsers, but converts P4 code into a TCAM-based implementation. The problem it solves is different from the problem described before: it compiles P4 parsing code to be used in hardware, while we convert sequential C-like code into a FSM (similar to the output of ParserHawk) and then back to P4. ParserHawk was very useful in inspiring ideas which shaped our project, mainly the synthesis encoding (Section 3.5) and logical specification construction.

3. Research Design and Implementation

3.1 DSL Design

We borrow ideas from C and P4 to design a DSL that is best suited for our purpose. A program in this DSL is stored in a `.parser` file.

The DSL allows a list of struct declarations at the top, each with a set of fields of type `bit_N`, where `N` is a positive integer denoting the bit-width of the field. This is very similar to the header declarations P4 allows.

Following the struct declarations, we declare the packet header vector as an anonymous struct in C with the fixed name `phv`. Its fields are headers whose types are structs declared earlier in the program. This corresponds to `headers_t` in P4.

Only one function is allowed: `int parse()`, which takes no arguments and contains the sequential C-like code representing the parser. It can be observed from Figure 1.1 that in the C code, a pointer is cast to a struct variable and then advanced. In place of this, we use a simplified statement `Extract(phv.<header_name>)` which signifies the same operation.

For `if-else` statements, we allow conditions of two kinds: exact matches, represented by `phv.<header>.<field> == <constant>`, and ternary matches, represented by `(phv.<header>.<field> & <mask>) == <value>`. No other forms are supported (no `&&`, `||`, or compound conditions).

Every path in `parse()` is required to end with `return ACCEPT` or `return REJECT`, mirroring the C convention of stating the final accept/reject decision.

As in C and P4, headers or header types which are not declared previously cannot be used. Such programs are invalid `.parser` code and are rejected.

The DSL equivalent of the C parser from Figure 1.1 is shown in Figure 3.1.

3.2 Pipeline Overview

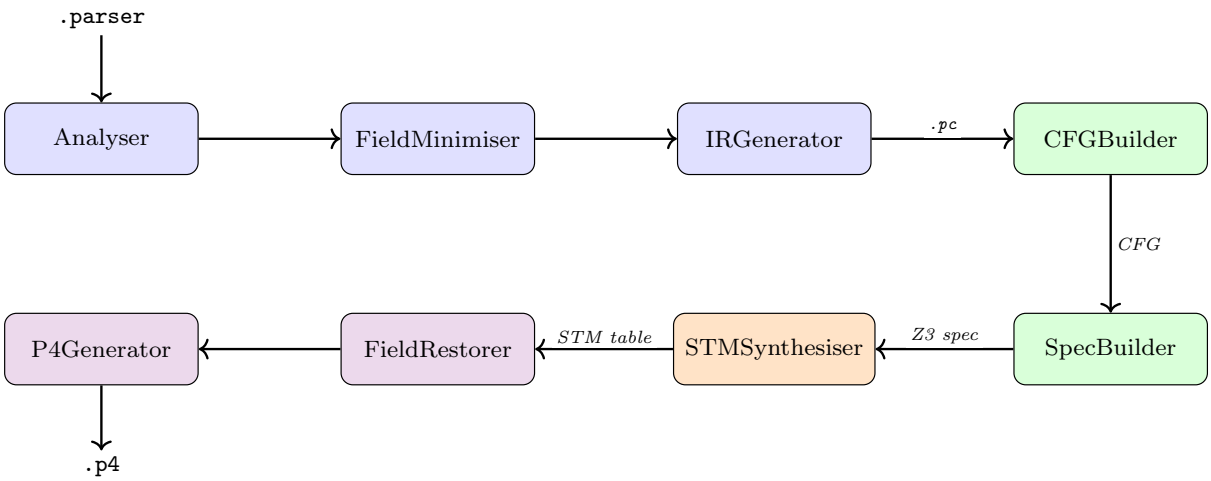

Figure 3.2: The synthesis pipeline. Boxes are colour-coded by phase: frontend (blue), specification construction (green), synthesis (orange), and backend (violet).

We use `pycparser` [3], a Python parser for C to parse the input `.parser` program. Since our DSL is a restricted subset of C, the AST produced by `pycparser` is a convenient starting point for the rest of the pipeline.

```
struct ethernet_t {
    bit_48 dst;
    bit_48 src;
    bit_16 ether_type;
};

struct ipv4_t { /* fields */ };
struct arp_t  { /* fields */ };

struct {
    ethernet_t ethernet;
    ipv4_t     ipv4;
    arp_t      arp;
} phv;

int parse() {
    Extract(phv.ethernet);

    if (phv.ethernet.ether_type == 0x0800) {
        Extract(phv.ipv4);
        return ACCEPT;
    } else if (phv.ethernet.ether_type ==
    0x0806) {
        Extract(phv.arp);
        return REJECT;
    }
    return ACCEPT;
}
```

Figure 3.1: The DSL equivalent of the C parser from Figure 1.1.

As shown in Figure 3.2, the pipeline is broadly split into four stages: a frontend, a specification-construction phase, the synthesis phase, and a backend. Each stage consists of a small number of modules.

Frontend

The frontend consists of the Analyser, FieldMinimiser, and IRGenerator. Together, they collect information about the program, apply field-size optimisations, and produce an IR for the next stages of the pipeline.

- **Analyser:** Traverses the AST and collects information about each struct in the program, the members of `phv` and the fields of each member. It also records whether each field is involved in any comparison and the type of the comparison. This is packaged in a class we call `Info`.
- **FieldMinimiser:** Operates on the `Info` produced by the Analyser, applies the optimisations described in Section 3.3.2 and returns an updated `Info`. This step is optional and the pipeline has been designed such that this module may be replaced to experiment with different optimisations.
- **IRGenerator:** Uses the updated `Info` together with another traversal over the AST to emit code in our intermediate representation, which we store in a `.pc` file.

Specification construction

This stage consists of the CFGBuilder and the SpecBuilder. It also collects information which is used in optimisations during synthesis.

- **CFGBuilder:** Constructs a CFG for the `.pc` IR by traversing the AST. It also rejects malformed programs. For example, those with a path that does not end in `ACCEPT` or `REJECT`.
- **SpecBuilder:** Walks over all possible paths in the CFG and generates a logical specification representing the parser, which is then used by the synthesiser.

Synthesis

The STMSynthesiser takes the specification produced by the SpecBuilder and synthesises the state machine. It uses a CEGIS loop and constrains the search to a tabular representation of the FSM. This restricted format is what allows the synthesis to remain tractable on realistic parsers.

Backend

The backend consists of two modules:

- **FieldRestorer:** Undoes the size and value transformations applied by the FieldMinimiser, so that the synthesised state machine refers to the original fields and constants from the input.
- **P4Generator:** Walks the restored state machine and emits the final P4 code, which is accepted as a valid parser by `p4c`.

3.3 Frontend

The frontend has three modules: Analyser, FieldMinimiser, and IRGenerator. Their roles have been summarised in Section 3.2.

3.3.1 Analyser

Apart from collecting `Info`, the Analyser validates input programs to ensure that they are well-formed. It rejects all programs that have one or more of the following bugs:

- Disallowed constructs
- Members of `phv` that are referenced in an `Extract` call or in a condition but were not declared in the `phv` struct
- Header types used in the `phv` declaration that were never declared as a struct earlier
- Redeclaration of a struct type, or of members within a struct or within `phv`
- Absence of the `int parse()` function, or presence of additional functions other than `parse`.
- `if` conditions that do not match either the exact form (`phv.<header>.<field> == <constant>`) or the ternary form (`(phv.<header>.<field> & <mask>) == <value>`).

A program that fails any of these checks raises with CompilationError.

3.3.2 FieldMinimiser

FieldMinimiser operates on the Info produced by the Analyser and aims to reduce field sizes as much as possible while making sure that the state machine produced by synthesis is either the same as the one we would have got without the optimisation, or any difference should be predictable and it can be corrected later during code generation.

The synthesis search space grows with the number of bits in each field, and also the size of the encoded constraints. The encoding of state transitions is linear in `max_packet_size`. Packet headers typically contain wide fields that play no role in branching: for instance, the 48-bit destination MAC in the Ethernet header is not used in any comparison. Maintaining their full widths unnecessarily enlarges the synthesis problem.

The restrictions on the DSL are what make these optimisations possible. In particular, the restriction that every condition is over a single field.

Every field in a header is either uncompared, compared only with exact matches (`field == value`), or compared with at least one ternary match. These three classes are handled separately.

Class 1: uncompared fields

There are two sub-cases:

- **The struct contains at least one compared field.** The uncompared fields are dropped for the sake of synthesis. The DSL always extracts a struct as a whole and the final P4 code does the same, so these fields need not be considered in the synthesiser.
- **The struct contains no compared fields.** Such a struct cannot be dropped directly: it still has to be extracted at runtime to advance the packet pointer correctly and dropping it would lose the evidence that the extraction actually happens. Instead, we treat the whole struct as a single 1-bit field during synthesis. Assigning the field to have a size of 0 is incorrect. An alternative approach, where such structs are dropped entirely for the synthesis and stitched back at code generation, is discussed in the FieldRestorer.

Class 2: fields with only exact matches

The actual values of the constants used in exact matches do not matter for synthesis. They are just decision points. If a field is compared against the constants $c_1, c_2, \ldots, c_n$, we can renumber them to $0, 1, \ldots, n-1$ for synthesis and undo the renumbering at code generation.

When branching on such a field there are at most $n+1$ outcomes: the field equals one of the n constants or it is not equal to any of them. All values not in $\{c_1, \ldots, c_n\}$ behave the same way in the FSM, so a single value is enough to represent them. Thus the field can be reduced to $\lceil \log_2(n+1) \rceil + 1$ bits during synthesis.

This renumbering is applied per field, which is correct because the DSL doesn't allow multi-field conditions.

Class 3: fields with ternary matches

Ternary matches are more difficult to handle. A single bit position can take part in more than one mask, so the renumbering approach from Class 2 does not apply. Swapping bit values would break some of the masked conditions.

We considered two heuristics, neither of which gives a strong guarantee:

- **Drop unmasked bits.** For each field, take the union of all bit positions that appear in any mask. Bits outside this union do not affect any condition, so they can all be collapsed into a single representative bit. This works well when masks are sparse but becomes ineffective when masks have many bits set.
- **Wildcard partitioning.** Express the field as a union of disjoint wildcard patterns and treat each pattern as a distinct symbolic constant. The number of patterns can grow quickly, so this does not scale.

We do not have a general bound on the reduction achieved by either heuristic. Ternary matches are rare in practical parsing programs, so a more advanced treatment is left for future work.

3.3.3 IRGenerator

The IRGenerator emits the IR as a `.pc` file instead of directly constructing the CFG. This is because the specification-construction and synthesis stages of the pipeline were written first and they used `.pc` as input and produced the state-machine output as a table. The frontend and the backend were written later. The IRGenerator emits `.pc` to avoid making unnecessary changes in the specification generation phase.

A `.pc` program is very similar to a `.parser` program except for the following changes in `.pc` programs:

- There are no structs. Each field of a `phv` member involved in an extraction or a check is assigned a distinct variable name `<phv_member>_<field>_<n>`, where `<n>` is a fresh number that keeps every name unique. This is needed to avoid two different fields having the same name in cases like `header1_`'s field `field0` and `header1`'s field `_field0`. Although such cases are rare, we preferred to have a correctness guarantee while renaming.
- The widths of the fields and the constants have been modified based by the Field-Minimiser.
- Each extract contains the size of the field being extracted along with the name

The `.pc` program corresponding to the running example (Figure 3.1) is shown in Figure 3.3.

```
int parse() {
    Extract(ethernet_ether_type_0,2);
    if (ethernet_ether_type_0 == 0) {
        Extract(ipv4_<first_field>_1,1);
        return ACCEPT;
    } else if (ethernet_ether_type_0 == 1) {
        Extract(arp_<first_field>_2,1);
        return REJECT;
    }
    return ACCEPT;
}
```

Figure 3.3: The `.pc` program produced by the IRGenerator for the running example in Figure 3.1.

The transformations that produced this output are:

- `ethernet`'s `dst` and `src` fields are uncompared and `ethernet` has a compared field, so they are dropped (Class 1a).
- `ether_type` is compared against two constants (`0x0800` and `0x0806`), which are renumbered to `0` and `1`. The field is shrunk from size `16` to size `2` (Class 2).
- `ipv4` and `arp` have no compared fields, so both are shrunk to single `1` bit fields named using their first declared field (Class 1b).
- All the above fields are named using their name and the `phv` member they belong to with a globally unique `<n>` suffix.

A drawback of not having structs in a `.pc` program is that when a `.parser` program has a `phv` member that has at least two fields which are used for comparisons, the `.pc` program would have to have separate extractions for these two fields. An idea to overcome this is considered below.

An alternative we considered

Instead of handling each field separately, we considered merging the whole header into one wide field and lifting all per-field comparisons into masked comparisons over this combined field. In principle, this could reduce the number of states needed.

This approach has two drawbacks:

- The number of distinct mask-value pairs needed can grow as $O(\prod_i k_i)$, where k_i is the number of conditions on the i-th subfield. This blows up quickly.
- It inherits the ternary handling problem from Class 3.

For these reasons, we did not pursue this approach.

3.4 Specification Construction

The specification-construction stage has two modules: CFGBuilder and SpecBuilder. Their roles have been summarised in Section 3.2.

3.4.1 CFGBuilder

Apart from constructing the CFG, the CFGBuilder also performs some additional validation on the input. It rejects programs which have any of the following bugs:

- Programs in which the same field is extracted more than once on some path from the entry of `parse()` to an exit.
- Programs that branch on a field along some path before that field has been extracted on the same path.

A program that fails any of these checks raises with `CompilationError`.

The CFG produced for the running example (Figure 3.3) is shown in Figure 3.4.

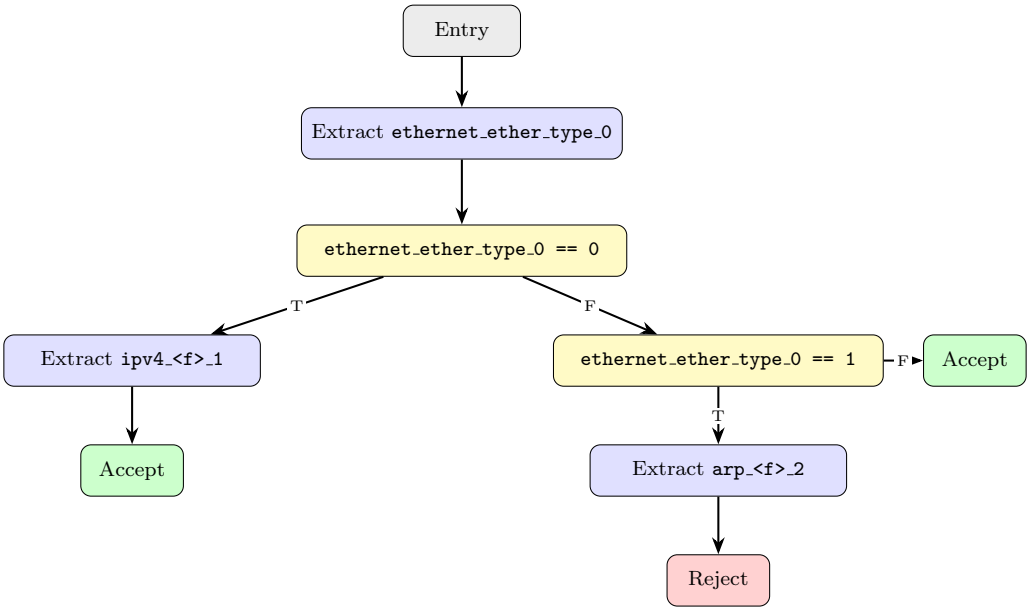

Figure 3.4: The CFG produced by the CFGBuilder for the `.pc` program in Figure 3.3. Colour key: blue = `ExtractNode`, yellow = `IfThenElseNode`, green = `Accept` exit, red = `Reject` exit.

3.4.2 SpecBuilder

We have borrowed ideas from the implementation of ParserHawk [4] for this module. The specification produced is a list of expressions for the fields and a single Boolean expression for the final accept/reject state as functions of the packet.

All fields are padded to `maximum_field_size+1` so that the expressions for all the fields have the same Z3 bit-vector type. The MSB is used to mark a field as "not extracted" on a path: the default value of a field is a bit-vector value with MSB set and every other bit unset. Since extracted bit-vectors have size at most `maximum_field_size` and are zero-extended, no extracted bit-vector can ever be equal to the default. This is useful in the synthesis stage.

The specification is built using a recursive walk over the CFG, starting at the entry node with a packet pointer at the first bit that hasn't been extracted. At every node we carry a current PHV (`cur_phv`). This is a list of expressions for the values of the fields on the current path. At the entry node, every entry of `cur_phv` is the default value.

The specification construction happens during backtracking:

- At an `ExitNode`, we return a copy of the default PHV along with the True/False bit recorded on this node.
- At an `ExtractNode` for field `fno` of size `sz`, we set

  ```
  cur_phv[fno] = ZeroExt(maximum_field_size+1-sz, Extract(ptr+sz-1, ptr,
      packet))
  ```

  and recurse on the next node with the packet pointer advanced by `sz`. When the recursion returns, we overwrite the field's entry in the returned PHV with the same expression, since the `ExitNode`s at the leaves return only the default values. It can be observed that no overwrite happens here, since no field is extracted twice on any path.
- At an `IfThenElseNode` branching on field `fno` with value `v`, we walk the true and false branches to obtain `(true_phv, true_state)` and `(false_phv, false_state)`.

For every field i, the i-th entry of the merged PHV is `If(cur_phv[fno] == v, true_phv[i], false_phv[i])` (or with a mask, for ternary matches). The accept/reject state is merged similarly.

The expressions returned at the root are the specification given to the synthesiser. For the running example (Figure 3.4), the specification is:

```
ether_type = ZeroExt(1, Extract(1, 0, packet))
ipv4       = If(ZeroExt(1, Extract(1, 0, packet)) == 0,
                ZeroExt(2, Extract(2, 2, packet)),
                default)
arp        = If(ZeroExt(1, Extract(1, 0, packet)) == 0,
                default,
                If(ZeroExt(1, Extract(1, 0, packet)) == 1,
                   ZeroExt(2, Extract(2, 2, packet)),
                   default))
state      = If(ZeroExt(1, Extract(1, 0, packet)) == 0,
                True,
                If(ZeroExt(1, Extract(1, 0, packet)) == 1,
                   False,
                   True))
```

where `default` is the bit-vector `100` in binary. The largest field after minimisation is of size 2, so `maximum_field_size+1` $= 3$ bits.

3.5 Synthesis

This is where the logical formula constructed in the previous phase is used to run a CEGIS loop and synthesise the state machine as a table.

The design of the state machine is inspired by the P4 parser state machine described in Figure 1.1. We considered two designs that differ in a small way. We describe the first design in detail in this section and briefly mention how the second design is different.

3.5.1 First Design

This design works only for programs with exact comparisons and is less general than the second design, but we describe it in detail because it motivates the second design and also enables a powerful optimisation in the FieldRestorer.

The state machine is expressed as a table. Each state has a default field which it extracts and branches on, and a default next state which is taken when none of the listed transitions are satisfied. Apart from this, there is a list of transition entries, each of the form `(current_state, match_value, next_state)`.

When making state transition, the field extracted in that state is compared against all the transition entries for that state. The matching entry is used to perform the transition. Since only exact comparisons are involved, at most one entry can match and if none of them matches the default transition is taken.

The synthesis itself is a CEGIS loop with two components: a Generator and a Verifier. The Generator produces state machines compliant with the counterexamples seen so far. The Verifier checks if it satisfies the specification for every possible input packet by trying to a counterexample. Each counterexample is added as a fresh correctness constraint to the Generator and the loop continues until the Verifier finds no counterexample for the synthesised state machine. The two have similar designs, so we describe the Generator only. We run a fresh CEGIS loop for each candidate `(table_size, no_states)` pair and take the first state machine the Verifier accepts as the output.

Generator

The Generator enforces three kinds of basic constraints on the state-machine table:

- **Domain constraints.** For example, the default field number of every state must be one of the allowed field numbers.
- **Determinism constraints.** No two transition entries from the same state may have the same matched value but different next states.
- **Ordering for optimisation.** A state transition can only go from a state with a lower number to a state with a higher number.

In addition to these structural constraints, the Generator encodes the symbolic simulation of the state machine and the per-packet correctness constraints.

We simulate the state machine by repeatedly applying the transition, starting from `cur_state = 0`, `cur_packet_ptr = 0`, and `cur_phv` set to the default values. The ordering constraint above forces the synthesised state machine to be loopless. We accept this restriction since the input programs are loopless as well. In such a state machine applying transitions for `no_states` steps is enough to reach accept or reject.

A transition takes `(cur_phv, cur_state, cur_packet_ptr)` to `(next_phv, next_state, next_packet_ptr)` parameterised by the table variables. All three components are nested `If` expressions. `cur_state` is integer-valued (the current state number), `cur_phv` is a list of bit-vector expressions for the fields and `cur_packet_ptr` is an integer expression giving the bit position of the next field to be extracted.

The transition is itself one nested `If` expression. It is built in four steps: field extraction, default transition, match lookup, and bounds check. These are executed in that order. Each step overrides the previous when its condition holds. Below, `sz` stands for `field_sizes[f]` and `M` for the padded width `maximum_field_size + 1`.

1. Field extraction. For every `(s, f, p)` with `p + sz` $\leq$ `max_packet_size`:

```
g = (cur_state == s) and (default_field[s] == f)
                     and (cur_packet_ptr == p)
next_packet_ptr = If(g, p + sz, next_packet_ptr)
next_phv[f]     = If(g, ZeroExt(M - sz,
                                Extract(p + sz - 1, p, packet)),
                        next_phv[f])
```

2. Default transition. `next_state` is initialised to `cur_state`, so accept and reject states don't transition. For every state `s`:

```
next_state = If(cur_state == s,
                default_next_state[s],
                next_state)
```

3. Match lookup. For every `(idx, s, f)`:

```
g = (cur_state == s) and (default_field[s] == f)
                     and (entry_state[idx] == s)
                     and (next_phv[f] == entry_match_value[idx])
next_state = If(g, entry_next_state[idx], next_state)
```

4. Bounds check. For every `(s, f, p)` with `p + sz > max_packet_size`: **change to ¿= here and ¡ previously ??**

```
g = (cur_state == s) and (default_field[s] == f)
                     and (cur_packet_ptr == p)
next_state = If(g, reject, next_state)
```

For each input packet (a bit-vector constant) added during the CEGIS loop, the correctness constraint checks whether the final state and PHV match the specification after `no_states` simulation steps.

Limitations of the first design

State replication. Since each field can be extracted only once on a path and must be branched on immediately, we end up replicating states unnecessarily. Consider:

```
Extract(field0, 4);
Extract(field1, 1);
if (field0 == 4) {
    ...
} else {
    ...
}
```

The state extracting `field1` has to be replicated, since the branch on `field0` has to happen first.

Priority among transitions (with ternary matches). We also tried to extend the first design to support ternary comparisons by adding a mask alongside the value in each transition entry. Exact comparisons would use a mask of size `maximum_field_size` with all bits set. This extension runs into a problem when the values matched by two ternary conditions overlap. Consider:

```
Extract(field0, 4);
if ((field0 & 3) == 1) {
    if ((field0 & 12) == 4) {
        ...
    }
    ...
}
...
```

The state extracting `field0` needs both a transition for the case where both masks are satisfied and a transition for the case where only the outer mask is satisfied. These cannot be encoded as transitions from a single state without enforcing a priority between them. The priority could be expressed naturally (transitions checked first having higher priority), but we did not pursue this. A state machine with priority-ordered transitions is not standard, and a parser is typically expected to have at most one matching transition at every state.

3.5.2 Second Design

We observed that the issues in the previous design came from allowing only the state which extracted a field to branch on it. In this design we relax this constraint. Every state must have a default field assigned to it like in the previous design and it may or may not extract it. It may or may not branch on it as well. This corresponds to the case where there is only a default transition.

Some small changes need to be made in the transitions. We now need to ensure two things: one is that while branching on a field, we do so only if it has been extracted, and the other is that while extracting, we do so only if it hasn't been extracted before.

The first is ensured in the match-lookup step of the transition. A table entry for a state is considered only if the default field's expression for that state is not equal to its default value, which can happen only if the field has been extracted on this path. If it is equal to the default, the transition based on it is not made and the entry is not used.

The second is ensured by checking, during an extract, that the field expression equals the default value. This can happen only when it has not been extracted before. The bounds-check step also performs a similar check so that a transition to reject is taken only when the state actually attempts an extraction.

The fact that the default value can never be taken by an extracted field (ensured during specification construction) is what makes these checks possible.

Since we iterate over the number of table entries and the number of states, we will not encounter table entries that don't make sense. If there is no table entry for a state, the default transition is taken on every visit. We ignore the default field in this case as it doesn't matter. We would also have to run an additional graph traversal algorithm to find the state which extracts the field after the synthesis as the state machine itself doesn't explicitly mention whether a particular state performs an extraction or not.

It can be seen that both of the problems of the original state machine are solved by this design. But it makes applying certain optimisations harder due to the extra flexibility.

Constant synthesis

If there are only exact matches in a program, we can collect the constants and constrain the values that the `match_value`s in the table can take, to reduce the search space. This can be done for ternary matches as well if we are following the second design. Both `match_value`s and `match_mask`s can be constrained to the constants and masks appearing in the program.

An observation on iteration speed

Running the synthesis from a smaller number of states and table entries to larger ones works faster than directly instantiating the larger problems, even when a new Z3 solver is created for each iteration. This is probably because of the CDCL(T) algorithm that Z3 runs. It caches some clauses in its memory across calls within the same process, which results in the speedup.

This is one of the reasons why we kept the iteration over table entries and states. The other is that it leads to a simpler transition encoding, without having to worry about inconsistent entries in the tables.

3.6 Backend

The backend has two modules, FieldRestorer and P4Generator, whose roles have been summarised in Section 3.2.

3.6.1 FieldRestorer

The FieldRestorer reverses the transformations done by the FieldMinimiser. The `Info` class produced by the frontend contains all the information needed for this. It remembers the original bit-widths of the fields and the renumbering of the constants. The FieldRestorer reads from `Info` and maps every entry of the synthesised state-machine table back to its original form.

3.6.2 P4Generator

The P4Generator walks the restored state machine and emits the final P4 code. P4 allows only the extraction of whole headers, not field by field. The synthesised state machine extracts one field per state, so the P4Generator groups the per-field extracts of the same

header into a single `packet.extract(hd.<header>)` call. This is done by a simple graph traversal that identifies, for each header, the state that extracts its first field. That state then emits the whole-header extract in the P4 output.

This grouping is especially important for the second design. In that design, fields of a header may not be extracted in the same order they appear in the source program, so the grouping is what reconstructs a valid P4 parser.

3.6.3 Drop-and-stitch optimisation

We discussed in Section 3.3.2 that headers which have none of their fields used in any comparison may be dropped completely from the synthesis and stitched back into the state machine afterwards. This is the optimisation referred to there. We believe it is easiest to apply with the first design.

Realistic parsers tend to stop branching after extracting transport protocol headers, and most of their conditions are exact matches. In such parsers, most of the headers are uncompared. Dropping them from the input before synthesis reduces the number of states the synthesiser has to deal with significantly.

Consider the parse function:

```
int parse() {
    Extract(phv.hdr0);
    Extract(phv.hdr1);
    Extract(phv.hdr2);
    if (phv.hdr0.field0 == 3) {
        if (phv.hdr1.field1 == 2) {
            Extract(phv.hdr4);
        }
    } else if (phv.hdr0.field0 == 4) {
        if (phv.hdr1.field1 == 5) {
            Extract(phv.hdr5);
        }
    }
    return ACCEPT;
}
```

where `hdr0.field0` and `hdr1.field1` are the only compared fields, and `hdr2`, `hdr4` and `hdr5` are uncompared. We drop `hdr2`, `hdr4` and `hdr5` from the input and run synthesis on the remainder. After this drop, all paths through the program reach Accept with the same observable behaviour, so the synthesiser produces the straight-line state machine in Figure 3.5.

Figure 3.5: The synthesised state machine after dropping `hdr2`, `hdr4` and `hdr5` from the input.

We now stitch the dropped headers back into this machine and add the missing transitions. From the CFG, different values of `hdr0.field0` lead to different sequences of further extractions, so the single outgoing transition out of the state extracting `hdr0.field0` is split into three: one each for `== 3`, `== 4` and default. The state extracting `hdr1.field1` is then replicated across the `== 3` and `== 4` branches, since the two branches go on to extract different headers (`hdr4` on one, `hdr5` on the other). `hdr2` appears in multiple places in the stitched machine for similar reasons. The final state machine is shown in Figure 3.6.

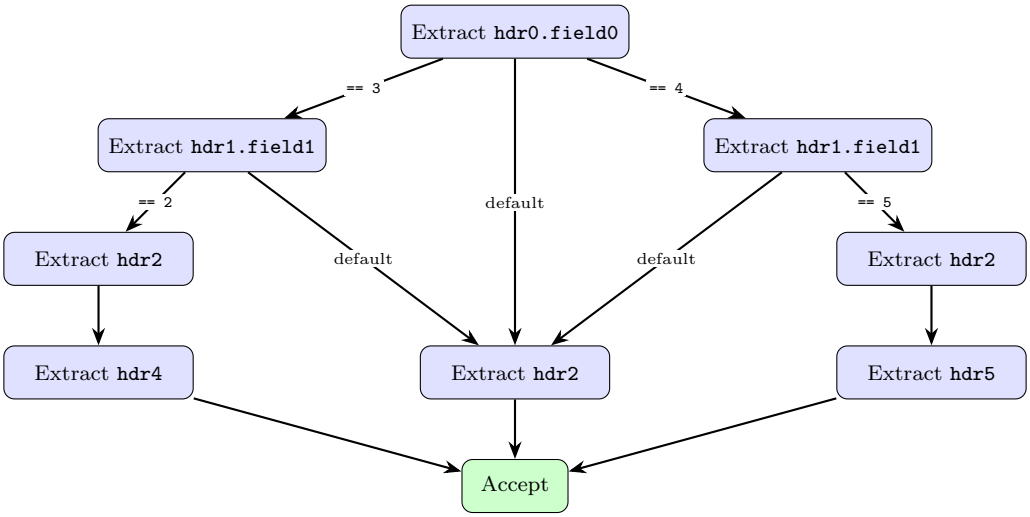

Figure 3.6: The state machine after stitching back `hdr2`, `hdr4` and `hdr5`. The state extracting `hdr2` on the default paths is shared.

This stitched state machine is equivalent to what the synthesiser would have produced if the uncompared headers had been kept, but is obtained much faster. The stitching procedure is mechanical and can be fully automated. It would be a useful extension to make the pipeline practical on larger parsers, where the synthesis problem with all fields kept becomes intractable.

4. Testing and Evaluation

We tested the design by running each configuration on a set of hand-written parsers and reviewing the synthesised state machine for correctness. A small Python helper renders the synthesised state machine as a PDF, which made the manual review easier.

We evaluate the design separately for input programs that use only exact comparisons and for programs that involve ternary comparisons since the supported configurations differ in the two cases.

4.1 Exact comparisons

We ran five benchmarks with each of the following configurations. `max_states` and `max_table_entries` were both set to ten. A timeout of 20 minutes was imposed on each run.

1. Design 1 with constant synthesis and field minimisation.
2. Design 1 with field minimisation only.
3. Design 1 with constant synthesis only.
4. Design 1 with no optimisation.

The same set of configurations was repeated for Design 2.

Field minimisation was the most effective optimisation. It reduced the synthesis time to roughly half of the unminimised baseline on most benchmarks. Constant synthesis also produced a reasonable speedup, though the effect was smaller and varied more across benchmarks. We may also observe that for `check_after_extract1`, the second design produced an FSM with one state fewer, as expected.

Table 4.1 reports the total field width across all PHV fields with and without field minimisation on these benchmarks. Table 4.2 reports the resulting state count, table-entry count and synthesis time for each configuration under both designs.

Benchmark	field min. on (bits)	field min. off (bits)
`linear`	3	48
`single_if`	3	40
`single_if_elif_else`	4	56
`multiple_if`	5	72
`check_after_extract1`	4	48
`check_after_extract2`	6	80

Table 4.1: Total field width across all PHV fields with and without field minimisation, on the exact-comparison benchmarks.

Benchmark	CS	FM	Design 1			Design 2		
			States	Entries	Time (s)	States	Entries	Time (s)
linear	on	off	3	0	1.23	3	0	2.44
linear	on	on	3	0	0.43	3	0	0.24
linear	off	off	3	0	1.44	3	0	1.84
linear	off	on	3	0	0.45	3	0	0.19
single_if	on	off	3	1	68.08	3	1	51.89
single_if	on	on	3	1	14.86	3	1	21.98
single_if	off	off	3	1	55.11	3	1	50.49
single_if	off	on	3	1	11.18	3	1	23.41
check_after_extract1	on	off	4	2	132.97	4	2	164.23
check_after_extract1	on	on	4	2	35.47	4	2	144.92
check_after_extract1	off	off	4	2	159.22	4	2	279.61
check_after_extract1	off	on	4	2	39.28	4	2	133.84
single_if_elif_else	on	off	5	3	669.05	5	3	631.46
single_if_elif_else	on	on	5	3	195.05	5	3	458.28
single_if_elif_else	off	off	5	3	866.20	-	-	timeout
single_if_elif_else	off	on	5	3	65.64	5	3	91.88
multiple_if	on	off	7	3	931.27	6	2	557.88
multiple_if	on	on	7	3	339.76	6	2	249.91
multiple_if	off	off	7	3	696.18	6	2	652.19
multiple_if	off	on	7	3	421.73	6	2	330.57

Table 4.2: Synthesis results on the exact-comparison benchmarks across all four CS/FM configurations, under both designs. CS stands for constant synthesis and FM for field minimisation. A dash indicates that the configuration timed out within 1200 s.

4.2 Ternary comparisons

Only Design 2 supports ternary comparisons, as discussed in Section 3.5, so we evaluated it with two configurations: field minimisation on, and field minimisation off. Field minimisation was applied only on the fields that participate in exact comparisons or in no comparison at all. Fields involved in ternary comparisons were left untouched.

A timeout of 20 minutes was imposed on each run. The benchmarks were run with eight states and eight table entries as the maximum. Table 4.3 reports the maximum bit width of a packet. Table 4.4 reports the resulting state count, table-entry count and synthesis time. A dash indicates that the configuration timed out.

Benchmark	field min. on (bits)	field min. off (bits)
`single_if_else`	6	12
`multiple_if`	9	28
`check_after_extract`	10	48
`nested_if_else2`	6	12
`nested_if_else1`	11	104

Table 4.3: Maximum bit width of a packet with and without field minimisation, on the ternary benchmarks.

Benchmark	Field min.	States	Entries	Time (s)
`single_if_else`	on	4	1	18.85
`single_if_else`	off	4	1	18.55
`multiple_if`	on	4	3	432.67
`multiple_if`	off	4	3	484.76
`check_after_extract`	on	4	2	79.64
`check_after_extract`	off	4	2	206.32
`nested_if_else2`	on	4	2	56.29
`nested_if_else2`	off	4	2	64.38
`nested_if_else1`	on	6	3	602.07
`nested_if_else1`	off	-	-	timeout

Table 4.4: Synthesis results on ternary benchmarks with Design 2. Field minimisation was applied only on fields that do not participate in ternary comparisons.

The state and table-entry counts match across the two configurations for every benchmark that completed and the run with field minimisation on runs faster always. The speedup from field minimisation is substantial on `check_after_extract` and `nested_if_else1`. On `nested_if_else1` the minimised configuration completes in about ten minutes while the unminimised one times out.

5. Conclusion and Future Work

We have thus built a pipeline which takes a C-like sequential packet parsing code and produces a state machine as a P4 parser using program synthesis. The source code is available at [5].

5.1 Limitations

Our design has a few limitations:

1. The DSL is very restrictive. It only allows extracts, no lookaheads, no conditions involving `&&` or `||`, and no cross-packet state. All of these would be desirable in some cases for packet processing.

2. We have ignored the difference between the byte order in network packets and the way structs are stored in memory.

3. Bitmask handling doesn't scale well. See the Class 3 case in Section 3.3.2 and the "alternative we considered" subsection in the IRGenerator.

4. The state machine produced doesn't necessarily have the minimum number of states.

5.2 Future Work

To build on this work:

1. Extend the design to overcome the limitations above.

2. Develop a more efficient method to handle bitmasks and automate the drop-and-stitch optimisation.

3. Investigate better synthesis analyses to improve performance and scalability.

4. Use these ideas for further analyses to enable vector processing, in particular handling the control flow divergence that arises due to parsing.

Bibliography

[1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proceedings of the 2013 Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–8. IEEE, 2013.

[2] Rajeev Alur, Rishabh Singh, Dana Fisman, and Armando Solar-Lezama. Search-based program synthesis. *Communications of the ACM*, 61(12):84–93, 2018.

[3] Eli Bendersky. pycparser: Complete c99 parser in pure python. `https://github.com/eliben/pycparser`, 2024. Accessed: 2026-05-03.

[4] Xiangyu Gao, Jiaqi Gao, Karan Kumar G, Muhammad Haseeb, Ennan Zhai, Bili Dong, Joseph Tassarotti, Srinivas Narayana, and Anirudh Sivaraman. ParserHawk: Hardware-aware parser generator using program synthesis. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25. ACM, 2025.

[5] Arya Krishnamurthy. Parsercompiler26: source code. `https://github.com/Aryakrish05/ParserCompiler26/tree/main`, 2026. Accessed: 2026-05-04.

[6] Microsoft. Z3 guide: Programming with Z3 in Python. `https://microsoft.github.io/z3guide/programming/Z3%20Python%20-%20Readonly/Introduction#arithmetic`, 2024. Accessed: 2026-05-01.

Acknowledgement of LLM Usage

LLMs were helpful in the final stages of this project. They were used in the following ways.

1. Formatting the report in LaTeX and producing the diagrams and tables in it.
2. Rapid prototyping of ideas added in the final stage, mainly to check the applicability of the FieldRestorer optimisations. The LLM was not able to produce a working implementation of the optimisation, and that code has not been added to the GitHub repository.
3. Writing PowerShell scripts to run the benchmarks.
4. Writing a few small functions in the final design that involved Python features I was not familiar with, such as the `re` module and some additional file-handling operations.
5. Generating the boilerplate code that follows the synthesised parser in the P4 output, such as the ingress and egress processing blocks.